INFO125 Practice Test

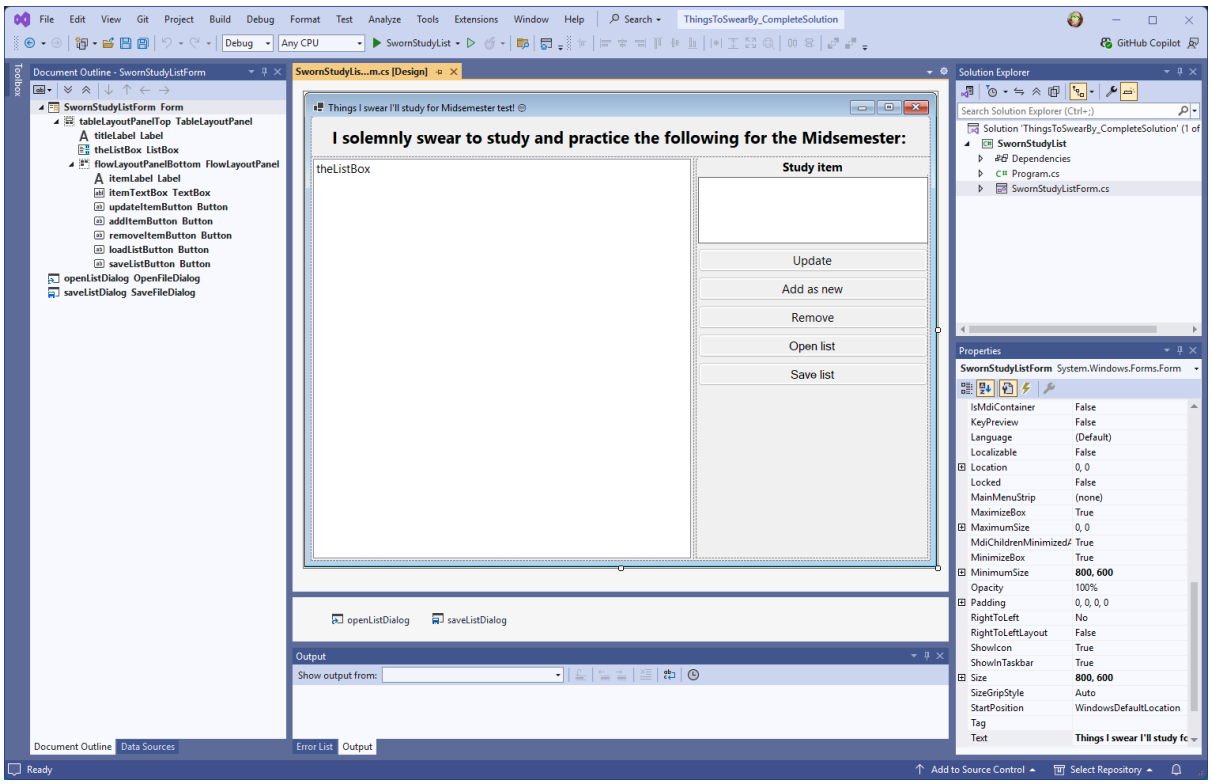
The Study Items List for the Midsemester Test

Introduction

This test focuses on the material covered in lectures and tutorials and Term One, including basic WinForms programming skills, writing event listeners, working with `ListBox` and `TextBox` controls, using `File.ReadAllLines()` and `File.WriteAllLines()` to read from and write to text files, transferring data between a string array and a `ListBox`, and applying simple selection and input validation in a small desktop application.

Setting Up

Download the Practice Test base code (`ThingsToSwearBy_PracticeTest.zip`) to your P drive, where you are sure you'll know to find it later. Extract the solution and open it in Visual Studio. It should look like this:

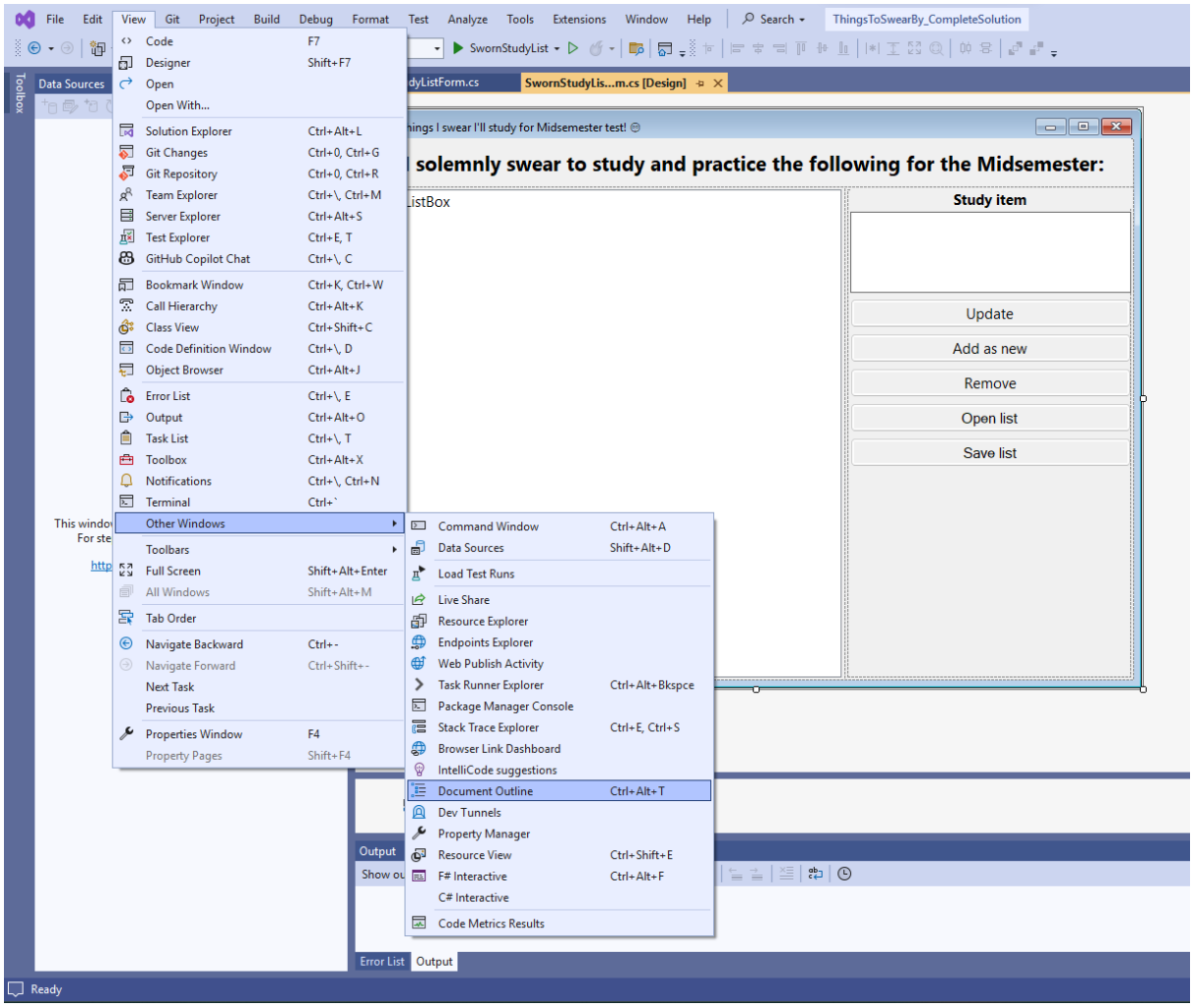


The form design, controls, and file dialogs are already provided in the solution – no controls need to be edited or added.

The solution folder also includes the `"Things I swear I'll study for Midsemester test.txt"`; examine this text file now and use it for testing purposes.

Make sure you can run the application. It should appear as a complete form, but the buttons do not yet have any functionality attached, apart from the `FormClosing` event for the form.

Examine the form, the controls, and note their names. A very helpful tool in this context is the Document Outline window, which can be unearthed like this:



Test Instructions

You are required to complete the missing code (event listeners) in the provided WinForms project, `SwornStudyList`, according to following instructions.

Overall, excluding blank lines and lines containing only braces, you don't need to write more than 40–45 lines of code.

Define and write the code for six event listener methods only:

1. `theListBox_SelectedIndexChanged`, for `theListBox` → `SelectedIndexChanged`[1] event
   Write code so that when the user selects (clicks on) an item in `theListBox`, the text of that selected item is displayed in `itemTextBox`; to achieve this, `theListBox.SelectedIndex`, which is automatically updated with every user interaction, can be used to get any of the items in

[1] https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.listbox.selectedindexchanged

the `theListBox.Items` array; look out for situations when the index is `-1` (e.g., when the form is first loaded) which means there is no current selection. The pseudocode for this method is this:

```
if theListBox.SelectedIndex is not -1
      use SelectedIndex value to get the relevant item from the listBox as String
      put it the itemTextBox.Text
```

(About three lines of code)

2. `loadListButton_Click`, for the `loadListButton` → `Click` event

   Write code so that when the user clicks `loadListButton`, a text file can be selected and opened, and all lines from that file are loaded into `theListBox`. Any existing items in `theListBox` should be cleared before the new items are loaded. Do not add the empty strings, which would later result in empty lines when the file is saved. It is suggested that you use the `File.ReadAllLines`[2] method to read a file and return an array of strings in one go, instead of reading the file line-by-line. (About seven lines of code excluding blank lines)

3. `saveListButton_Click`, for the `saveListButton` → `Click` event

   Write code so that when the user clicks `saveListButton`, the current items in `theListBox` are saved to a text file. Each item in the list should be written as a separate line in the file. It is suggested that you use the `File.WriteAllLines` method to write an array of strings to a file in one go, instead of writing to file line-by-line[3]. (About 10 lines of code excluding blank lines)

4. `addToListButton_Click`, for the `addToListButton` → `Click` event

   Write code so that when the user clicks `addToListButton`, the text currently entered in `itemTextBox` is added to `theListBox` as a new item. Do not add the item if the text box is empty, which would later result in empty lines when the contents is written to a file. (About 10 lines of code excluding blank lines)

5. `updateItemButton_Click`, for the `updateItemButton` → `Click` event

   Write code so that when the user clicks `updateItemButton`, the selected item in `theListBox` is replaced with the text currently entered in `itemTextBox`. If no item is selected, display an appropriate message. Do not update the item if the text box is empty, which would result in empty lines when the contents is written to a file. Here we need to rely on the `theListBox.SelectedIndex` to tell `theListBox` which item to update/replace. (About 10 to 12 lines of code excluding blank lines)

6. `removeItemButton_Click`, for the `removeItemButton` → `Click` event

   Write code so that when the user clicks `removeItemButton`, the selected item is removed from `theListBox`. If no item is selected, display an appropriate message. Again, we rely on the `theListBox.SelectedIndex` to tell `theListBox` which item to remove. (About 10 lines of code excluding blank lines)

---

[2] https://learn.microsoft.com/en-us/dotnet/api/system.io.file.readalllines
[3] https://learn.microsoft.com/en-us/dotnet/api/system.io.file.writealllines

Wrapping Up

Ensure your project/solution compiles.

Perform thorough testing to convince yourself the application works as expected.

Marvel at your achievements!